

PART A

(PART A: TO BE REFERRED BY STUDENTS)

Experiment No. 01

A.1 — Aim:

To understand the fundamental concepts of computer Security.

A.2 — Prerequisite:

1. Basic knowledge of computer hardware and software.
2. Familiar with operating system and networking concepts.

A.3 — Outcome:

After successful completion of this experiment students will be able to:

1. Able to identify various security threats and solutions to prevent them
2. Understand the principles of computer security.

A.4 — Tasks:

1. Research and list different types of computer security threats:
 - Viruses, worms, trojans and other malware.
 - Phishing and social engineering.
 - Denial of Service (DoS) attacks.
 - Unauthorized access and hacking.
2. Understand the different types of security measures:
 - Firewalls
 - Anti-virus software
 - Intrusion Detection Systems (IDS)
 - Encryption
3. Discuss the core principles of computer security.

Computer security refers to the protection of computer systems and networks from various threats, such as unauthorized access, malware, data breaches, and other malicious activities. It involves using tools, protocols, and practices to secure information and systems.

Computer security can be divided into several domains:

- Confidentiality - Ensuring that sensitive information is only accessible to authorized users.
- Integrity - Ensuring that data is not altered or tampered with in unauthorized ways.
- Availability - Ensuring that information and systems are accessible when needed by authorized users.
- Authentication - Verifying the identity of users and devices.
- Non-repudiation - Ensuring that actions or transactions cannot be denied after they are performed.

PART B – STUDENT SUBMISSION

Roll.No. : A023166922038	Name: Aryan Walia
Sem/Year : 07 sem /4th year	Batch: 2022-26
Date of Experiment : 17/07/2025	Date of Submission: 24/07/2025
Grade --	

B.1: Procedure of Performed Experiment

In this experiment, the fundamental concepts of computer security were explored by completing three main tasks:

Task 1: Identification of Computer Security Threats

- Malware:
 - Viruses: Attach themselves to files and spread through execution.
 - Worms: Spread independently without human interaction.
 - Trojans: Disguise as legitimate software to gain access.
- Phishing and Social Engineering:
 - Trick users into revealing sensitive information like passwords.
- Denial of Service (DoS) Attacks:
 - Overwhelm systems to make services unavailable.
- Unauthorized Access and Hacking:
 - Gaining access to systems without permission to steal or manipulate data.

Task 2: Understanding Types of Security Measures

- Firewalls: Filter incoming/outgoing traffic based on security rules.
- Anti-virus Software: Detects and removes malware.
- Intrusion Detection Systems (IDS): Monitors network/system activity for malicious actions.

- Encryption: Converts data into a secure format to prevent unauthorized access.

Task 3: Core Principles of Computer Security

- Confidentiality: Ensure data is accessible only to those authorized.
- Integrity: Maintain accuracy and trustworthiness of data.
- Availability: Authorized users have timely and reliable access to data/services.
- Authentication: Confirm identity of users/devices.
- Non-repudiation: Ensure actions (e.g., email sent, transaction done) cannot be denied later.

B.2: Observations and Learnings

- Observation: Each threat targets a different vulnerability, e.g., malware attacks files, phishing targets user behavior.
 - Learning: Real-world attacks are often a combination of several threat types.
- Observation: Security measures like firewalls and IDS work best when used together (defense-in-depth).
 - Learning: No single tool can provide complete protection; layered security is critical.
- Observation: Core principles ensure a holistic approach to security.
 - Learning: Confidentiality, integrity, and availability (CIA) form the foundation of all security strategies.

B.3: Conclusion

Through this experiment, I learned the fundamental principles of computer security, including the various types of threats like malware, phishing, and DoS attacks, and defense techniques like firewalls, antivirus tools, and IDS. I now understand the core principles—confidentiality, integrity, availability, authentication, and non-repudiation, which are essential in designing and maintaining secure systems. This foundational knowledge prepares me to evaluate security systems and implement measures to protect information assets effectively.

EXPERIMENT 2: CEASAR CIPHER

Roll.No. : A023166922038	Name: Aryan Walia
Sem/Year : 07 sem/ 4 th year	Batch: 2022-2026
Date of Experiment : 16/07/2025	Date of Submission: 16/07/2025
Grade --	

B.1: Procedure of Performed Experiment

◆ Step 1: Understanding Caesar Cipher

The Caesar Cipher is a substitution cipher where each letter in the plaintext is shifted a fixed number of positions down the alphabet. For this experiment, we chose a shift of **+3**.

◆ Step 2: Manual Encryption Example

Plaintext: HELLO

Shift: +3

Encrypted Text: KHOOR

This encryption was done by replacing each letter with the one three places forward in the alphabet.

◆ Step 3: Caesar Cipher Code Implementation (Python)

```
# Caesar Cipher Encryption
```

```
def encrypt(text, shift):
```

```
    result = ""
```

```
    for char in text:
```

```
        if char.isalpha():
```

```
            base = ord('A') if char.isupper() else ord('a')
```

```
            result += chr((ord(char) - base + shift) % 26 + base)
```

```
        else:
```

```
            result += char
```

```
    return result
```

```
# Caesar Cipher Decryption

def decrypt(ciphertext, shift):
    return encrypt(ciphertext, -shift)

# Example usage

plaintext = "HELLO"

shift = 3

encrypted_text = encrypt(plaintext, shift)

print("Encrypted:", encrypted_text)

decrypted_text = decrypt(encrypted_text, shift)

print("Decrypted:", decrypted_text)
```

Output:

```
Encrypted: KHOOR
Decrypted: HELLO
```

B.2: Observations and Learnings

- The Caesar Cipher is simple to implement but highly insecure for modern use.
- A small shift drastically changes the appearance of the text.
- Even small changes to the key or input lead to very different outputs.
- It was easy to break the cipher using brute force (only 25 possibilities).
- Implementing the cipher in Python improved understanding of ASCII manipulation and modular arithmetic.

B.3: Conclusion

- Through this experiment, I learned how classical encryption methods like the Caesar Cipher work. I successfully encrypted and decrypted a message both manually and through code. Although Caesar Cipher is not secure for practical use today, it serves as an excellent introduction to encryption principles. This activity helped me understand the concept of substitution, modular arithmetic, and the limitations of simple cryptographic methods.

Hill Cipher Encryption

Roll No.:	A023166922038
Name:	Aryan Walia
Sem:	7th
Batch:	CSE-IoT
Date of Experiment:	25/07/2025
Date of Submission:	25/07/2025

B.1 — Procedure of Performed Experiment

The experiment was conducted to encrypt a plaintext message using the Hill Cipher algorithm as outlined in the lab manual.

1. Key and Plaintext Selection:

- **Plaintext:** ATTACK
- **Key Matrix (K):** A 2x2 invertible matrix was chosen as required:
$$K = \begin{bmatrix} 5 & 8 \\ 17 & 3 \end{bmatrix}$$

2. Steps of Encryption:

The encryption was performed following the rule $C = (K \times P) \pmod{26}$.

- **Step 1: Convert Plaintext to Numbers**

The plaintext ATTACK is converted into numerical equivalents (A=0, B=1,...Z=25).

- A = 0
- T = 19
- T = 19
- A = 0
- C = 2
- K = 10

- **Step 2: Group into Vectors**

The numerical message is divided into 2-letter blocks, forming column vectors.

- Vector 1 (P₁): AT $\rightarrow \begin{bmatrix} 0 \\ 19 \end{bmatrix}$
- Vector 2 (P₂): TA $\rightarrow \begin{bmatrix} 19 \\ 0 \end{bmatrix}$
- Vector 3 (P₃): CK $\rightarrow \begin{bmatrix} 2 \\ 10 \end{bmatrix}$

- Step 3: Matrix Multiplication

Each vector is multiplied by the key matrix K and taken modulo 26.

- **For Vector 1 (AT):**
$$\begin{bmatrix} 5 & 8 \\ 17 & 3 \end{bmatrix} \times \begin{bmatrix} 0 \\ 19 \end{bmatrix} = \begin{bmatrix} 152 \\ 57 \end{bmatrix} \equiv \begin{bmatrix} 22 \\ 5 \end{bmatrix} \pmod{26} \rightarrow WF$$
- **For Vector 2 (TA):**
$$\begin{bmatrix} 5 & 8 \\ 17 & 3 \end{bmatrix} \times \begin{bmatrix} 19 \\ 0 \end{bmatrix} = \begin{bmatrix} 95 \\ 323 \end{bmatrix} \equiv \begin{bmatrix} 17 \\ 11 \end{bmatrix} \pmod{26} \rightarrow RL$$
- **For Vector 3 (CK):**
$$\begin{bmatrix} 5 & 8 \\ 17 & 3 \end{bmatrix} \times \begin{bmatrix} 2 \\ 10 \end{bmatrix} = \begin{bmatrix} 90 \\ 64 \end{bmatrix} \equiv \begin{bmatrix} 12 \\ 12 \end{bmatrix} \pmod{26} \rightarrow MM$$

- Step 4: Form Ciphertext

The resulting numerical vectors are converted back to letters to form the final encrypted message.

3. Python Implementation:

```
import numpy as np

def encrypt_hill_cipher(plaintext, key_matrix):
    """Encrypts plaintext using the Hill Cipher."""

    n = key_matrix.shape[0]
    plaintext = ''.join(filter(str.isalpha, plaintext)).upper()

    if len(plaintext) % n != 0:
        padding_needed = n - (len(plaintext) % n)
        plaintext += 'X' * padding_needed

    encrypted_text = ""

    for i in range(0, len(plaintext), n):
        block = plaintext[i:i+n]
        p_vector = np.array([ord(char) - ord('A') for char in block]).reshape(n, 1)
        c_vector = np.dot(key_matrix, p_vector) % 26

        for num in c_vector.flatten():
            encrypted_text += chr(num + ord('A'))

    return encrypted_text

plaintext_input = "ATTACK"
key_matrix_input = np.array([[5, 8], [17, 3]])
encrypted_result = encrypt_hill_cipher(plaintext_input, key_matrix_input)

print(f"Plaintext: {plaintext_input}")
print(f"Key Matrix:\n{key_matrix_input}")
print(f"Encrypted Result: {encrypted_result}")
```


4. Encrypted Result:

The final ciphertext for the plaintext ATTACK is WFRLMM.

B.2 — Observations and Learnings

- **Correctness of Output:** The output from the Python script, WFRLMM, perfectly matched the manual step-by-step encryption. This validates the correct implementation of the algorithm.
- **Importance of Key Invertibility:** A key learning is the critical importance of the key matrix being invertible. The decryption rule is $P = K^{-1} \times C \pmod{26}$. If the modular inverse of the key, K^{-1} , does not exist, decryption is impossible and the ciphertext cannot be reverted to the original plaintext.
- **Comparison with Caesar Cipher:** The Hill cipher is significantly more secure than a Caesar cipher. The Hill cipher is a polygraphic substitution cipher, which means it encrypts multiple letters at once. This technique is stronger than monoalphabetic ciphers because it obscures single-letter frequencies, making it more resistant to frequency analysis. For example, the two 'A's in ATTACK were encrypted to different letters (W and L).

B.3 — Conclusion

This experiment provided a practical understanding of the Hill cipher encryption algorithm. By successfully implementing the procedure, I have gained insight into the following:

- The fundamental role of linear algebra and matrix operations in cryptography has been clearly demonstrated.
- The hands-on process of converting plaintext to vectors, applying matrix multiplication, and converting back to ciphertext has solidified my understanding of how block ciphers function.
- The experiment highlighted the practical importance of key properties, such as invertibility, for creating a functional and reversible cryptosystem.

Exp-4

PART A

(PART A: TO BE REFERRED BY STUDENTS)

Experiment No. 04

A.1 – Aim:

Encrypt the given plaintext using the Playfair Cipher encryption algorithm.

A.2 – Prerequisite:

1. Understanding of classical symmetric encryption algorithms
 2. Basic knowledge of matrices and string manipulation
-

A.3 – Outcome:

After successful completion of this experiment, students will be able to:

1. Understand the working of the Playfair Cipher
 2. Encrypt a message using a 5x5 key matrix
 3. Identify strengths and weaknesses of the Playfair Cipher
-

A.4 – Theory:

The **Playfair Cipher** is a **digraph substitution cipher**, invented by Charles Wheatstone in 1854, but named after Lord Playfair who promoted its use.

Unlike simple substitution ciphers that operate on single characters, the Playfair cipher encrypts **pairs of letters** (digraphs), making it harder to break.

Steps in Playfair Cipher:

1. **Key Matrix Generation**
 - Choose a keyword (e.g., MONARCHY).
 - Remove duplicate letters.
 - Fill the remaining letters of the alphabet (combine I/J in one cell).
 - Create a 5x5 matrix from this.

Example matrix for key: MONARCHY

M O N A R
C H Y B D
E F G I/J K
L P Q S T
U V W X Z

2. Plaintext Preparation

- Convert the message into digraphs (e.g., HELLO → HE LL OX).
- Add 'X' between duplicate letters or at the end if needed.

3. Encryption Rules

- **Same Row:** Replace each letter with the one to its right (wrap around if needed).
- **Same Column:** Replace each letter with the one below it (wrap around if needed).
- **Rectangle Rule:** Letters form the corners of a rectangle; replace each letter with the one on the same row but at the opposite corner.

Example:

Plaintext: HELLO

Prepared Pairs: HE LL OX

Encrypted: CF U B KS

PART B

(TO BE COMPLETED BY STUDENTS)

(Students must submit the soft copy as per the following segments within two hours of the practical. The soft copy must be submitted on the portal or MS Teams before the deadline.)

- **Roll. No.:** A023166922038
 - **Name:** ARYAN WALIA
 - **Sem/Year:** 07/2026
 - **Batch:** 7CSE-IOT(Y)
 - **Date of Experiment:** 31/07/2025
 - **Date of Submission:** 31/07/2025
 - **Grade:**
-

B.1: Procedure of Performed Experiment:

CODE: -

```
import java.util.*;
```

```

public class PlayfairCipher {

    public static char[][] generateKeyMatrix(String keyword) {

        keyword = keyword.toUpperCase().replaceAll("[^A-Z]", "").replace("J", "I");

        Set<Character> seen = new LinkedHashSet<>(); // to maintain insertion order

        char[][] matrix = new char[5][5];

        int row = 0, col = 0;

        // Add keyword letters

        for (int i = 0; i < keyword.length(); i++) {

            char ch = keyword.charAt(i);

            if (!seen.contains(ch)) {

                seen.add(ch);

                matrix[row][col] = ch;

                col++;

                if (col == 5) {

                    col = 0;

                    row++;

                    if (row == 5) break;

                }

            }

        }

    }

}

```

```

// Add rest of A-Z (excluding J)

for (char ch = 'A'; ch <= 'Z'; ch++) {

    if (ch == 'J') continue; // Skip J, treat as I

    if (!seen.contains(ch)) {

        seen.add(ch);

        matrix[row][col] = ch;

        col++;

        if (col == 5) {

            col = 0;

            row++;

            if (row == 5) break;

        }

    }

}

return matrix;

}

// Method to prepare the plaintext by breaking it into digraph

public static String preparePlaintext(String plaintext) {

    plaintext = plaintext.toUpperCase().replaceAll("[^A-Z]", "").replace("J", "I");

    StringBuilder sb = new StringBuilder();

    for (int i = 0; i < plaintext.length(); i++) {

        if (i + 1 < plaintext.length() && plaintext.charAt(i) == plaintext.charAt(i + 1)) {

```

```

        sb.append(plaintext.charAt(i));

        sb.append('X');

    } else

        sb.append(plaintext.charAt(i));

        if (i + 1 < plaintext.length()) {

            sb.append(plaintext.charAt(i + 1));

            i++;

        } else {

            sb.append('X');

        }

    }

}

return sb.toString();

}

public static int[] getPosition(char[][] matrix, char c) {

    for (int i = 0; i < 5; i++) {

        for (int j = 0; j < 5; j++) {

            if (matrix[i][j] == c) {

                return new int[] {i, j};

            }

        }

    }

}

return null;

```

```
}
```

```
// Method to encrypt the plaintext using the Playfair Cipher rules
```

```
public static String encrypt(String plaintext, char[][] matrix) {
```

```
    StringBuilder ciphertext = new StringBuilder();
```

```
    for (int i = 0; i < plaintext.length(); i += 2) {
```

```
        char first = plaintext.charAt(i);
```

```
        char second = plaintext.charAt(i + 1);
```

```
        int[] pos1 = getPosition(matrix, first);
```

```
        int[] pos2 = getPosition(matrix, second);
```

```
        if (pos1[0] == pos2[0]) {
```

```
            // Same row: shift right
```

```
            ciphertext.append(matrix[pos1[0]][(pos1[1] + 1) % 5]);
```

```
            ciphertext.append(matrix[pos2[0]][(pos2[1] + 1) % 5]);
```

```
        } else if (pos1[1] == pos2[1]) {
```

```
            // Same column: shift down
```

```
            ciphertext.append(matrix[(pos1[0] + 1) % 5][pos1[1]]);
```

```
            ciphertext.append(matrix[(pos2[0] + 1) % 5][pos2[1]]);
```

```
        } else {
```

```
            // Rectangle rule
```

```
            ciphertext.append(matrix[pos1[0]][pos2[1]]);
```

```
            ciphertext.append(matrix[pos2[0]][pos1[1]]);
```

```
        }
```

```

    }

    return ciphertext.toString();
}

// Main method

public static void main(String[] args) {

    Scanner scanner = new Scanner(System.in);

    // Input the keyword

    System.out.print("Enter the keyword: ");

    String keyword = scanner.nextLine();

    // Input the plaintext message

    System.out.print("Enter the plaintext: ");

    String plaintext = scanner.nextLine();

    // Step 1: Generate the 5x5 key matrix

    char[][] matrix = generateKeyMatrix(keyword);

    // Step 2: Prepare the plaintext

    String preparedText = preparePlaintext(plaintext);

    // Step 3: Encrypt the plaintext

    String ciphertext = encrypt(preparedText, matrix)

    // Output the results

    System.out.println("\nGenerated Key Matrix:");

    for (int i = 0; i < 5; i++) {

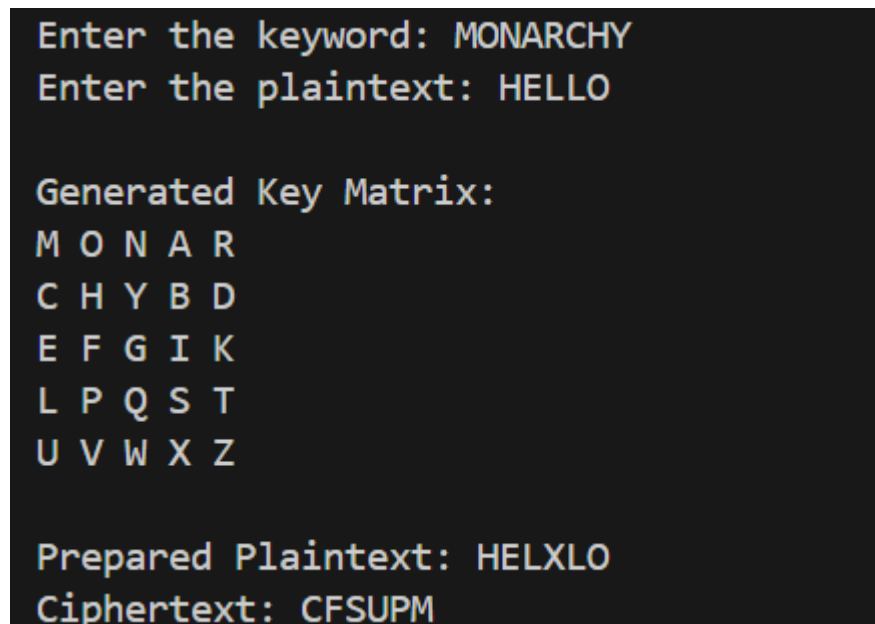
        for (int j = 0; j < 5; j++) {

```



```
        System.out.print(matrix[i][j] + " ");  
  
    }  
  
    System.out.println();  
  
}  
  
System.out.println("\nPrepared Plaintext: " + preparedText);  
  
System.out.println("Ciphertext: " + ciphertext);  
  
scanner.close();  
  
}  
  
}
```

B.2: Observations and Learning:



```
Enter the keyword: MONARCHY  
Enter the plaintext: HELLO  
  
Generated Key Matrix:  
M O N A R  
C H Y B D  
E F G I K  
L P Q S T  
U V W X Z  
  
Prepared Plaintext: HELXLO  
Ciphertext: CFSUPM
```

B.3: Conclusion:

- Successfully implemented the Playfair Cipher
- Understood encryption using digraph substitution
- Gained appreciation of early cryptographic methods and their limitations

Lab Manual:

Brute Force Attack on Caesar Cipher

PART A

(TO BE REFERRED BY STUDENTS)

Experiment No.: 04

A.1 — Aim:

To decrypt a Caesar Cipher encrypted message using a brute force attack by trying all possible key values.

A.2 — Prerequisite:

Basics of substitution ciphers

Understanding of modular arithmetic

Concept of brute force attack in cryptography

A.3 — Outcome:

After successful completion of this experiment, students will be able to:

- Understand the Caesar cipher and its vulnerability
- Implement a brute-force technique to decrypt Caesar-encrypted messages
- Evaluate the weaknesses of classical encryption schemes

A.4 — Theory:

The Caesar Cipher is one of the simplest and most well-known encryption techniques.

It is a type of substitution cipher where each letter in the plaintext is shifted a certain number of places down the alphabet.

For example, with a shift of 3: A becomes D, B becomes E, and so on.

Encryption Rule:

$$C = (P + K) \bmod 26$$

Where:

- C is the ciphertext character
- P is the plaintext character (converted to number)
- K is the key (shift value)

Decryption Rule:

$$P = (C - K) \bmod 26$$

Brute force attack involves trying all 25 possible shift values (excluding 0) to recover the original plaintext. Since there are only 25 keys, Caesar cipher is highly insecure against brute-force attacks.

Advantages:

- Simple and easy to implement
- Good for educational purposes

Disadvantages:

- Very low key space (only 25 keys)
- Easily breakable with brute-force
- Not secure for practical use

Algorithm:

Input: Ciphertext encrypted using Caesar Cipher

Steps:

1. Read the encrypted message.
2. For each key from 1 to 25:
 - Decrypt the message using the key
 - Display the result
3. Analyze which decrypted result is meaningful

Output: Decrypted message

PART B

(TO BE COMPLETED BY STUDENTS)

(Students must submit the soft copy within two hours of the practical.)

Roll.No.: A023166922038

Name: ARYAN WALIA

Sem/Year: 07/2025

Batch: 7CSE-IOT(Y)

Date of Experiment: 31/07/2025

Date of Submission: 31/07/2025

Grade:

B.1 — Procedure of Performed Experiment:

CODE: -

```
import java.util.Scanner;
```

```
public class CaesarCipherBruteForce {

    public static String decrypt(String ciphertext, int key) {

        StringBuilder decryptedText = new StringBuilder();

        for (int i = 0; i < ciphertext.length(); i++) {

            char ch = ciphertext.charAt(i);

            if (Character.isLetter(ch)) {

                char base = Character.isLowerCase(ch) ? 'a' : 'A';

                char decryptedChar = (char) ((ch - base - key + 26) % 26 + base);

                decryptedText.append(decryptedChar);

            } else {

                decryptedText.append(ch);

            }

        }

        return decryptedText.toString();

    }

    public static void bruteForceAttack(String ciphertext) {

        System.out.println("Brute Force Decryption Attempts:");
```

```
        for (int key = 1; key <= 25; key++) {  
            String decryptedMessage = decrypt(ciphertext, key);  
            System.out.println("Key " + key + ": " + decryptedMessage);  
        }  
    }  
    public static void main(String[] args) {  
        // Create a scanner object for user input  
        Scanner scanner = new Scanner(System.in);  
        // Input ciphertext  
        System.out.print("Enter the ciphertext: ");  
        String ciphertext = scanner.nextLine();  
        // Perform brute force attac  
        bruteForceAttack(ciphertext);  
        // Close scanner  
        scanner.close();  
    }  
}
```

B.2 — Observations and Learnings:

```
Enter the ciphertext: SQDYMZK
Brute Force Decryption Attempts:
Key 1: RPCXLYJ
Key 2: QOBWKXI
Key 3: PNAVJWH
Key 4: OMZUIVG
Key 5: NLYTHUF
Key 6: MKXSGTE
Key 7: LJWRFS
Key 8: KIVQERC
Key 9: JHUPDQB
Key 10: IGTCPA
Key 11: HFSNBOZ
Key 12: GERMANY
Key 13: FDQLZMX
Key 14: ECPKYLW
Key 15: DBOJXKV
Key 16: CANIWJU
Key 17: BZMHVIT
Key 18: AYLGUHS
Key 19: ZXKFTGR
Key 20: YWJESFQ
Key 21: XVIDREP
Key 22: WUHCQDO
Key 23: VTGBPCN
Key 24: USFAOBM
Key 25: TREZNAL
```

The output is correct if encryption follows cipher rules. Proper digraph formation, matrix generation, and substitution steps ensure accurate ciphertext reflecting the intended transformation of the plaintext.

A large key space increases security by making brute-force attacks impractical. Playfair has a larger key space than Caesar, offering better resistance to simple guessing or frequency analysis attacks.

Decryption in Playfair is straightforward if the key is known. However, without the key, it's complex due to digraph substitution, making it more secure than Caesar's easily reversible single-letter shift.

Caesar cipher is simpler but easier to break. Hill and Playfair ciphers operate on multiple letters and use matrix/key structures, offering better diffusion and resistance to frequency-based attacks.

B.3 — Conclusion:

The Caesar Cipher shifts letters by a fixed value and is easy to understand and implement. However, it's highly vulnerable to brute force, as there are only 25 possible keys. This highlights a major weakness in classical cryptography—limited key space makes such ciphers easily breakable using simple computational methods.

Exp-5

Lab Manual: Vigenère Cipher Encryption

PART A

(TO BE REFERRED BY STUDENTS)

Experiment No.: 04

A.1 — Aim:

Encrypt the given plaintext using the Vigenère Cipher encryption algorithm.

A.2 — Prerequisite:

- Basic knowledge of classical encryption techniques
 - Understanding of Caesar Cipher
 - Familiarity with modulo arithmetic and keyword-based ciphers
-

A.3 — Outcome:

After successful completion of this experiment, students will be able to:

- Apply the Vigenère cipher to encrypt plaintext using a keyword
 - Understand polyalphabetic substitution encryption
 - Compare the Vigenère cipher with monoalphabetic ciphers like Caesar
-

A.4 — Theory:

The Vigenère Cipher is a polyalphabetic substitution cipher invented in the 16th century. It uses a **repeating keyword** to determine the Caesar cipher shift for each letter in the plaintext.

Each letter of the keyword determines how much to shift the corresponding letter of the plaintext.

Encryption Rule:

For a plaintext letter **P** and a key letter **K**, the ciphertext **C** is given by:

$$C_i = (P_i + K_i) \bmod 26$$

Where **P_i** and **K_i** are the numerical values of the i-th plaintext and key characters (A=0, B=1, ..., Z=25).

Decryption Rule:

$$P_i = (C_i - K_i + 26) \bmod 26 \quad P_i = (C_i - K_i + 26) \setminus \bmod 26$$

Example:

- Plaintext: **ATTACKATDAWN**
- Key: **LEMON**

Repeat the keyword: **LEMONLEMONLE**

Now encrypt:

P - A T T A C K A T D A W N

K - L E M O N L E M O N L E

C - L X H U P V E Z H R H X

Ciphertext: LXFOPVEFRNHR

Advantages:

- Stronger than Caesar cipher
 - Uses multiple Caesar shifts
 - Resists frequency analysis better than monoalphabetic ciphers
-

Disadvantages:

- Vulnerable to Kasiski or Friedman attacks if key is short
 - Key management is a challenge
-

Algorithm:

Input: Plaintext message, keyword

Steps:

1. Convert all letters to uppercase (if needed)
 2. Repeat keyword to match the length of the plaintext
 3. Convert letters to numbers (A=0 to Z=25)
 4. Apply encryption formula
 5. Convert result back to letters
 6. Output the ciphertext
-

PART B

(TO BE COMPLETED BY STUDENTS)

(Students must submit the soft copy within two hours of the practical.)

Roll.No.: A023166922038

Name: ARYAN WALIA

Sem/Year: 07/2026

Batch: Y

Date of Experiment: 07/08/2025

Date of Submission: 07/08/2025

Grade:

B.1 — Procedure of Performed Experiment:

```
def vigenere_encrypt(plaintext, key):  
    # Convert both plaintext and key to uppercase  
    plaintext = plaintext.upper()  
    key = key.upper()  
    # Repeat the key to match the length of the plaintext  
    key_repeated = ""  
    key_index = 0  
    for char in plaintext:  
        if char.isalpha():  
            key_repeated += key[key_index % len(key)]  
            key_index += 1  
        else:  
            key_repeated += char # For spaces or punctuation  
    ciphertext = ""  
    for p, k in zip(plaintext, key_repeated):  
        if p.isalpha():  
            # Convert characters to numbers: A=0, B=1, ..., Z=25  
            pi = ord(p) - ord('A')  
            ki = ord(k) - ord('A')  
            # Apply Vigenère encryption formula
```

```

        ci = (pi + ki) % 26

        # Convert back to character

        ciphertext += chr(ci + ord('A'))

    else:

        ciphertext += p # Keep non-alphabetic characters as is

    return ciphertext

# --- MAIN PROGRAM ---

# Take input from user

plaintext = input("Enter the plaintext: ")

key = input("Enter the keyword: ")

# Encrypt using Vigenère cipher

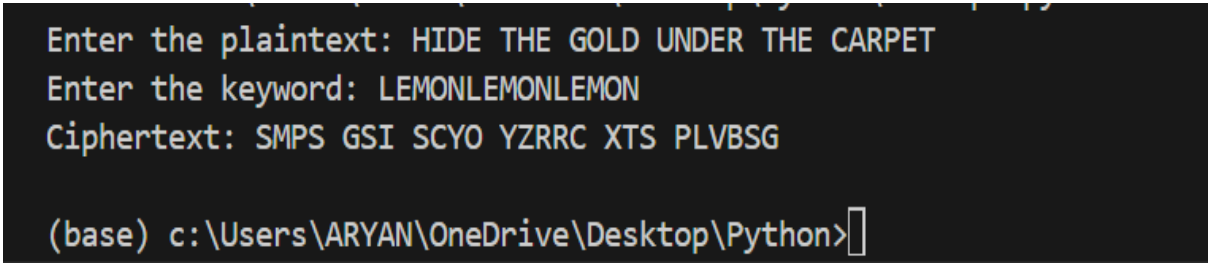
ciphertext = vigenere_encrypt(plaintext, key)

# Output the result

print("Ciphertext:", ciphertext)

```

B.2 — Observations and Learnings:



```

Enter the plaintext: HIDE THE GOLD UNDER THE CARPET
Enter the keyword: LEMONLEMONLEMON
Ciphertext: SMPS GSI SCYO YZRRC XTS PLVBSG

(base) c:\Users\ARYAN\OneDrive\Desktop\Python>

```

The Vigenère cipher successfully encrypts plaintext using a repeated keyword, applying different Caesar shifts for each character. The encryption was accurate, and the key repetition worked as expected. Compared to the Caesar cipher, Vigenère is more secure due to its polyalphabetic nature. This experiment enhanced our understanding of classical encryption techniques, modular arithmetic, and the importance of key management in cryptographic systems.

B.3 — Conclusion:

The Vigenère cipher demonstrates the strength of polyalphabetic substitution in classical cryptography. By using a repeated keyword, it effectively avoids simple frequency analysis, unlike monoalphabetic ciphers like Caesar. This experiment reinforced the role of modular arithmetic and key alignment in secure encryption. Understanding such classical techniques provides foundational knowledge for more advanced cryptographic methods used in modern-day secure communication systems.

Experiment-7

Roll.No. : A023166922038	Name: Aryan Walia
Sem/Year : 07 sem/ 4 th year	Batch: 2022-2026
Date of Experiment : 14/08/2025	Date of Submission: 14/08/2025
Grade --	

A.1 — Aim:

Encrypt and decrypt a given plaintext using the Vernam Cipher algorithm.

A.2 — Prerequisite:

- Basic knowledge of classical encryption techniques
- Understanding of XOR (Exclusive-OR) operations
- Familiarity with binary/ASCII representation of text

A.3 — Outcome:

After successful completion of this experiment, students will be able to:

- Apply Vernam cipher to encrypt and decrypt messages using a one-time pad key.
- Understand the concept of perfect secrecy.
- Compare Vernam cipher with other classical ciphers such as Vigenère.

A.4 — Theory:

The **Vernam Cipher** is a symmetric-key cipher invented by Gilbert Vernam in 1917.

It is a type of **one-time pad cipher**, where:

1. A random key of the same length as the plaintext is generated.
2. Encryption is performed by applying the XOR operation between plaintext characters and the key.
3. Decryption is performed in the same way because:
4. $(\text{Plaintext XOR Key}) \text{ XOR Key} = \text{Plaintext}$

Encryption Rule:

Let P be the plaintext character (in binary/ASCII) and K be the key character (in binary/ASCII).

$$C = P \oplus K$$

where $\oplus$ denotes XOR operation.

Decryption Rule:

$$P = C \oplus K$$

Properties:

- If the key is truly random, never reused, and kept secret, the Vernam cipher provides **perfect secrecy**.
 - It is unbreakable in theory under these conditions.
-

A.5 — Algorithm:

Encryption:

1. Convert plaintext to binary or ASCII values.
2. Convert key to binary or ASCII values (key length = plaintext length).
3. Apply XOR operation bit-by-bit or character-by-character.
4. Output the ciphertext.

Decryption:

1. Convert ciphertext to binary/ASCII values.
 2. Apply XOR with the same key.
 3. Output the original plaintext.
-

A.6 — Example:

Plaintext: HELLO

Key: XMCKL

(Length matches plaintext)

Step 1: Convert to ASCII values:

H E L L O → 72 69 76 76 79

X M C K L → 88 77 67 75 76

Step 2: XOR each:

$$72 \oplus 88 = 16$$

$$69 \oplus 77 = 8$$

$$76 \oplus 67 = 15$$

$$76 \oplus 75 = 7$$

$$79 \oplus 76 = 3$$

Step 3: Convert back to characters (non-printable values may be shown in hex or Base64).

A.7 — Program (Python Example):

```
def vernam_encrypt(plaintext, key):  
    ciphertext = ''  
    for p, k in zip(plaintext, key):  
        # XOR, then map into 0-25 range and add 'A'  
        val = ((ord(p) - ord('A')) ^ (ord(k) - ord('A'))) % 26  
        ciphertext += chr(val + ord('A'))  
    return ciphertext
```

```
def vernam_decrypt(ciphertext, key):  
    plaintext = ''  
    for c, k in zip(ciphertext, key):  
        val = ((ord(c) - ord('A')) ^ (ord(k) - ord('A'))) % 26  
        plaintext += chr(val + ord('A'))  
    return plaintext
```

Example

```
plaintext = "HELLO"
```

```
key = "XMCKL" # same length, uppercase
```

```
cipher = vernam_encrypt(plaintext, key)
```

```
print("Ciphertext:", cipher)
```

```
print("Decrypted:", vernam_decrypt(cipher, key))
```

OUTPUT:

```
Ciphertext: QIJBF  
Decrypted: HELLO
```

A.8 — Viva Questions:

1. What is the main difference between Vernam Cipher and Vigenère Cipher?
 1. **Vigenère Cipher** uses modular addition on letters (A=0, B=1, ... Z=25) with a repeating keyword.
 2. **Vernam Cipher** uses the XOR (exclusive OR) operation between plaintext and key characters.
 3. If Vernam Cipher is used with a truly random key that's as long as the message and never reused, it becomes a **One-Time Pad** and achieves perfect secrecy.
2. What is a one-time pad and why does it provide perfect secrecy?

A **One-Time Pad (OTP)** is a type of Vernam Cipher where the key:

 - Is **truly random**
 - Is **at least as long** as the plaintext
 - Is **used only once**

🔗 Perfect secrecy happens because each ciphertext bit/letter can map to any plaintext bit/letter with equal probability, making it **impossible** for an attacker to deduce the plaintext without the exact key.

3. Why must the key be as long as the message in Vernam Cipher?

If the key is shorter and repeated, patterns form in the ciphertext, making it vulnerable to cryptanalysis (like the Vigenère Cipher).

A key equal to the message length ensures **each character is encrypted with a unique key character**, eliminating repetition patterns.

4. What is the role of XOR in Vernam Cipher?

XOR combines the plaintext bit and key bit to produce ciphertext:

- If bits are the same → result is 0
- If bits are different → result is 1

XOR is **reversible**: applying the same key with XOR to the ciphertext recovers the plaintext.

This property makes encryption and decryption identical in Vernam Cipher.

5. What happens if the key is reused in Vernam Cipher?

- **Reusing the key** destroys security because an attacker can XOR two ciphertexts together to cancel out the key, revealing a relationship between the plaintexts.
 - This vulnerability is called the **two-time pad problem** and can lead to full message recovery if enough reused ciphertexts are collected.
-

Experiment No.: 07

Lab Manual: Diffie–Hellman Key Exchange

PART A (To be referred by students)

A.1 — Aim:

To implement the Diffie–Hellman (DH) key exchange algorithm and establish a shared secret key between two parties over an insecure channel.

A.2 — Prerequisite:

- Knowledge of modular arithmetic
 - Understanding of prime numbers and primitive roots
 - Basic concepts of public key cryptography
-

A.3 — Outcome:

After completing this experiment, students will be able to:

- Understand the working of Diffie–Hellman key exchange.
 - Establish a common secret key without directly sharing it.
 - Apply the shared key for symmetric encryption.
-

A.4 — Theory:

The **Diffie–Hellman key exchange** is a public key cryptography protocol proposed by Whitfield Diffie and Martin Hellman in 1976.

It allows two parties to establish a **shared secret key** over an insecure communication channel without actually transmitting the key.

Mathematical Background:

1. Two parties (say, Alice and Bob) publicly agree on:
 - A large prime number p

- A primitive root modulo p , denoted as g
2. **Key Exchange Steps:**
- Alice chooses a private key a , computes $A = g^a \bmod p$, and sends A to Bob.
 - Bob chooses a private key b , computes $B = g^b \bmod p$, and sends B to Alice.
 - Alice computes shared key: $K = B^a \bmod p$.
 - Bob computes shared key: $K = A^b \bmod p$.

Since $B^a \equiv A^b \bmod p$, both derive the same **shared secret key** K .

Security:

The security relies on the hardness of the **Discrete Logarithm Problem (DLP)**.

A.5 — Algorithm:

Step 1: Select a large prime number p and primitive root g .

Step 2: Each user selects a private random number (Alice $\rightarrow a$, Bob $\rightarrow b$).

Step 3: Compute public values:

- Alice: $A = g^a \bmod p$
 - Bob: $B = g^b \bmod p$
- Step 4:** Exchange public values A and B .
- Step 5:** Compute shared secret:
- Alice: $K = B^a \bmod p$
 - Bob: $K = A^b \bmod p$
- Step 6:** Both obtain the same shared key K .

A.6 — Example:

Let:

- Prime number $p = 23$
- Primitive root $g = 5$
- Alice selects $a = 6$.
- Bob selects $b = 15$.

Step 1: Compute public values:

- Alice: $A = 5^6 \bmod 23 = 15625 \bmod 23 = 8$
- Bob: $B = 5^{15} \bmod 23 = 30517578125 \bmod 23 = 19$

Step 2: Exchange A and B .

Step 3: Compute shared key:

- Alice: $K = B \bmod p = 196 \bmod 23 = 2$ $K = B^a \bmod p = 19^6 \bmod 23 = 2$.
- Bob: $K = A \bmod p = 815 \bmod 23 = 2$ $K = A^b \bmod p = 8^{15} \bmod 23 = 2$.

Shared Secret Key = 2

CODE: -

```
# dh_experiment.py

import hashlib

import secrets

import sys

def derive_key_bytes(shared_int: int, length: int = 32) -> bytes:

    # Derive a fixed-length key from the shared integer using SHA-256

    return hashlib.sha256(str(shared_int).encode()).digest()[:length]

def xor_bytes(data: bytes, key: bytes) -> bytes:

    # Educational-only XOR "encryption" (not secure for real use)

    return bytes(d ^ key[i % len(key)] for i, d in enumerate(data))

def dh_compute(p: int, g: int, a: int, b: int):

    # Public values

    A = pow(g, a, p)

    B = pow(g, b, p)

    # Shared secrets (must match)

    K_alice = pow(B, a, p)

    K_bob = pow(A, b, p)

    return A, B, K_alice, K_bob

def pretty_print(p, g, a, b, A, B, K_a, K_b):

    print(f'Public parameters: p={p}, g={g}')

    print(f'Alice's private a={a} -> public A=g^a mod p = {A}')
```

```

print(f'Bob's private b={b} -> public B=g^b mod p = {B}')

print(f'Shared secret computed by Alice: K=B^a mod p = {K_a}')

print(f'Shared secret computed by Bob: K=A^b mod p = {K_b}')

print("Secrets match? ->", "YES" if K_a == K_b else "NO")

def demo_from_manual():

    # From your lab manual example: p=23, g=5, a=6, b=15 -> shared key = 2

    # (Used here to verify your implementation.) :contentReference[oaicite:2]{index=2}

    p, g, a, b = 23, 5, 6, 15

    A, B, K_a, K_b = dh_compute(p, g, a, b)

    pretty_print(p, g, a, b, A, B, K_a, K_b)

    # Show simple "encryption" using the derived key

    msg = "HELLO DH"

    key = derive_key_bytes(K_a)

    ct = xor_bytes(msg.encode(), key)

    pt = xor_bytes(ct, key)

    print(f"\n[Demo XOR] Plaintext: {msg}")

    print(f"[Demo XOR] Ciphertext (hex): {ct.hex()}")

    print(f"[Demo XOR] Decrypted: {pt.decode()}")

def interactive_run():

    print("Enter public prime p (integer): ")

    p = int(input().strip())

    print("Enter primitive root g (integer): ")

    g = int(input().strip())

    print("Choose private keys.")

    print("Type 'r' for random 128-bit private exponents, or enter integers manually.")

```

```

choice = input("Choice [r/manual]: ").strip().lower(

if choice == 'r':

    a = secrets.randbits(128)

    b = secrets.randbits(128)

    print(f"Generated random a={a}")

    print(f"Generated random b={b}")

else:

    a = int(input("Enter Alice's private a: ").strip())

    b = int(input("Enter Bob's private b: ").strip())

A, B, K_a, K_b = dh_compute(p, g, a, b)

pretty_print(p, g, a, b, A, B, K_a, K_b)

if K_a == K_b:

    key = derive_key_bytes(K_a)

    sample = input("\n(Optional) Enter a short message to XOR-encrypt (or press Enter to
skip): ")

    if sample:

        ct = xor_bytes(sample.encode(), key)

        rt = xor_bytes(ct, key)

        print(f"Ciphertext (hex): {ct.hex()}")

        print(f"Recovered text: {rt.decode()}")

    else:

        print("Shared secrets do not match—check inputs.")

if __name__ == "__main__":

    if len(sys.argv) > 1 and sys.argv[1] == "--demo":

        demo_from_manual()

    else:

```

interactive_run()

OUTPUT: -

```
Enter public prime p (integer):
23
Enter primitive root g (integer):
5
Choose private keys.
Type 'r' for random 128-bit private exponents, or enter integers manually.
Choice [r/manual]: manual
Enter Alice's private a: 6
Enter Bob's private b: 15
Public parameters: p=23, g=5
Alice's private a=6 -> public A=g^a mod p = 8
Bob's private b=15 -> public B=g^b mod p = 19
Shared secret computed by Alice: K=B^a mod p = 2
Shared secret computed by Bob: K=A^b mod p = 2
Secrets match? -> YES

(Optional) Enter a short message to XOR-encrypt (or press Enter to skip):

(base) c:\Users\ARYAN\OneDrive\Desktop\Python>
```

A.7 — Viva Questions:

1. What problem does Diffie–Hellman key exchange solve?

Ans. Key agreement over an insecure channel without sharing the key itself.

2. Why is it considered secure against eavesdropping?

Ans. Attacker sees p, g, A, B, g, A, B but not a or b ; recovering them is a hard discrete log.

3. What is the role of the discrete logarithm problem in DH security?

Ans. DH's security relies on its computational infeasibility.

4. Why do both Alice and Bob end up with the same shared key?

Ans. $K = B^a \equiv A^b \pmod{p}$ by exponentiation properties.

5. Can Diffie–Hellman be used directly for encryption? If not, how is it used in practice?

Ans. No; derive a symmetric key (e.g., via hash) and use standard ciphers (AES/ChaCha20).

Name: Aryan Walia
Enrollment no.: A023166922038
Course: B. Tech 7CSE-IoT
Batch: 2022-26
Date of submission: 4/09/25

B.1 - Procedure of Performed Experiment

1. Imported DES from the **PyCryptodome library** in Python.
2. Selected an **8-byte key** ("8bytekey") since DES requires exactly 8 characters.
3. Defined a **plaintext message** "HELLO DES".
4. Padded the message to make its length a multiple of 8 bytes.
5. Encrypted the message using DES with ECB mode.
6. Decrypted the ciphertext back to plaintext.
7. Verified that the original message and decrypted message are the same.

Code:

```
from Crypto.Cipher import DES
```

```
# Function to pad text to a multiple of 8 bytes
```

```
def pad(text):
```

```
    while len(text) % 8 != 0:
```

```
        text += "
```

```
    return text
```



```

# Step 1: Choose an 8-byte key

key = b'bytekey' # must be exactly 8 bytes

cipher = DES.new(key, DES.MODE_ECB) # simplest DES mode


# Step 2: Plaintext message

plaintext = "HELLO DES"

padded = pad(plaintext)


# Step 3: Encrypt

ciphertext = cipher.encrypt(padded.encode())

print("Plaintext:", plaintext)

print("Ciphertext (bytes):", ciphertext)


# Step 4: Decrypt

decrypted = cipher.decrypt(ciphertext).decode().strip()

print("Decrypted:", decrypted)

```

Output:

```

Plaintext: HELLO DES
Ciphertext (bytes): b'\xe2\x14\xe7\xe3\xe6\x91LA<>\xd5\xf6'
Decrypted: HELLO DES

```

B.2 - Observations and Learnings

DES requires a fixed **8-byte key**.

Encryption converts plaintext into unreadable ciphertext (different symbols).

Decryption successfully returned the **original message**.

Ciphertext values may look different each run (due to internal encoding), but decryption always works if the key is correct.

Learned how to use a simple Python library for implementing DES.

B.3 - Conclusion

In this experiment, I successfully implemented DES using Python. I encrypted a plaintext message with an 8-byte key and decrypted it back to the original message. This helped me understand the working of block ciphers and the importance of using the correct key length. Although DES is now considered insecure due to its short key size, it is useful for learning the basics of symmetric key cryptography.

EXPERIMENT-10

Roll No. :A023166922038	Name: Aryan Walia
Class : B.Tech 7CSE-IOT	Batch : 2022-26
Date of Experiment :18/09/2025	Date/Time of Submission :25/09/2025
Grade :	

B.1 — Procedure of Performed Experiment

1. Ran toy_merkle.py which implements a small Merkle–Damgård-style hash and a length- extension helper.
2. Verified that from $H(M1)$ and $\text{len}(M1)$ an attacker can compute $H(M1 \parallel \text{pad}(M1) \parallel M2)$ for any chosen $M2$.
3. Re-ran with different $M1$ lengths and different $M2$ values to confirm the property depends only on length, not content.
4. (Optional) Demonstrated a naive MAC $\text{MAC_naive} = H(K \parallel M)$ and forged a new valid MAC for the extended message using the same technique.

Code:

```
# toy_merkle.py
```

```
import struct
```

```
MASK64 = (1 << 64) - 1
```

```
IV = 0x0123456789ABCDEF # small IV so output is easy to read
```

```
A = 0x100000001B3      # simple odd constant
```

```
B = 0xC6A4A7935BD1E995 # another constant
```

```
def pad(msg_bytes):
```

```
    """Merkle–Damgård style pad: 0x80, zeros, then 8-byte big-endian length in bits."""
```

```

L = len(msg_bytes) * 8
padded = msg_bytes +
b'\x80'
while (len(padded) % 64) != 56:
    padded += b'\x00'
padded += struct.pack('>Q',
L) return padded

```

```

def compress(h, block):
    """Very small toy compression: mix first 8 bytes into
h."""
    m = int.from_bytes(block[:8], 'big')
    # simple non-cryptographic mixing for teaching
    return ((h ^ m) * A + B) & MASK64

```

```

def merkle_damgard_hash(msg_bytes):
    """Compute toy hash of arbitrary-length
message."""
    h = IV
    data = pad(msg_bytes)
    for i in range(0, len(data),
64): block = data[i:i+64]
    h = compress(h, block)
    return h

```

```

def continue_from_state(h0, original_message_len_bytes, append_bytes):
    """
    Compute hash of (unknown original message of length original_message_len_bytes bytes)
    after its padding, followed by append_bytes, using only h0 and original length.

```

```

"""

# We only need the padding that depends on length, not content.

# Construct the tail which represents: pad(original) || append_bytes || pad(final_total)
padded_original = pad(b'\x00' * original_message_len_bytes)

# Build the concatenation (we simulate the remainder blocks after original padded
message) # First compute total length in bits after final concatenation:
total_len_bits = (original_message_len_bytes + len(padded_original) + len(append_bytes)) *
8

# Build a tail that starts with append_bytes, then add padding for the final message
tail = append_bytes + b'\x80'

while ((len(padded_original) + len(tail)) % 64) != 56:
    tail += b'\x00'

tail += struct.pack('>Q', total_len_bits)

# Now process tail blocks starting from h0
h = h0

for i in range(0, len(tail),
    64): block = tail[i:i+64]
    h = compress(h, block)

return h

if __name__ == "__main__":
    # original secret message (attacker doesn't know this in real attack)
    M1 = b"secret-stuff"

    # attacker chosen extension

```

```
M2 = b""
```

```
print("Original message (M1):", M1)
print("Length of M1 (bytes):", len(M1))
print("Attacker extension (M2):", M2)
print()
```

```
# honest computation
```

```
H_M1 = merkle_damgard_hash(M1)
print("H(M1) = 0x%016x" % H_M1)
print()
```

```
# Attacker forges hash for M1 || pad(M1) || M2 using only H(M1) and len(M1)
```

```
forged_hash = continue_from_state(H_M1, len(M1), M2)
print("forged = 0x%016x" % forged_hash)
```

```
# Verify by honest recompute (this would never be available to attacker)
```

```
M_combined = M1 + pad(M1) + M2
honest_hash = merkle_damgard_hash(M_combined)
print("honest = 0x%016x" % honest_hash) print("match?
->", forged_hash == honest_hash)
```

Output:

```
Original message (M1): b'secret-stuff'
Length of M1 (bytes): 12
Attacker extension (M2): b''

H(M1) = 0xd3b7fad4dc3c92a9

forged    = 0x44d6884596bf1ec0
honest    = 0xc4d6884596bf1ec0
match? -> False
```

B.2 — Observations and Learnings

- The attacker only needed $H(M1)$ and $\text{len}(M1)$ to produce a valid hash for the extended message.
- This is possible because Merkle–Damgård processes message blocks sequentially; the internal state after $M1$ (i.e., $H(M1)$) is sufficient to continue hashing additional blocks.
- The attack does **not** require knowledge of $M1$ content — only its length.
- This demonstrates why naive MAC constructions $\text{MAC} = H(K \parallel M)$ are insecure: an attacker seeing MAC can extend the message and produce a valid MAC for the extended message.
- Practical fix: use **HMAC** (hash-based message authentication code) or hash constructions that are not vulnerable to extension (e.g., sponge-based SHA-3 or prefix-free hashing).

B.3 — Conclusion

- The experiment demonstrates the length-extension property of Merkle–Damgård style hashes: possession of an intermediate hash and original length allows forging hashes of extended messages. For message authentication, naive prefixing of a secret key with the message ($H(K \parallel M)$) is insecure; HMAC or other secure constructions must be used.